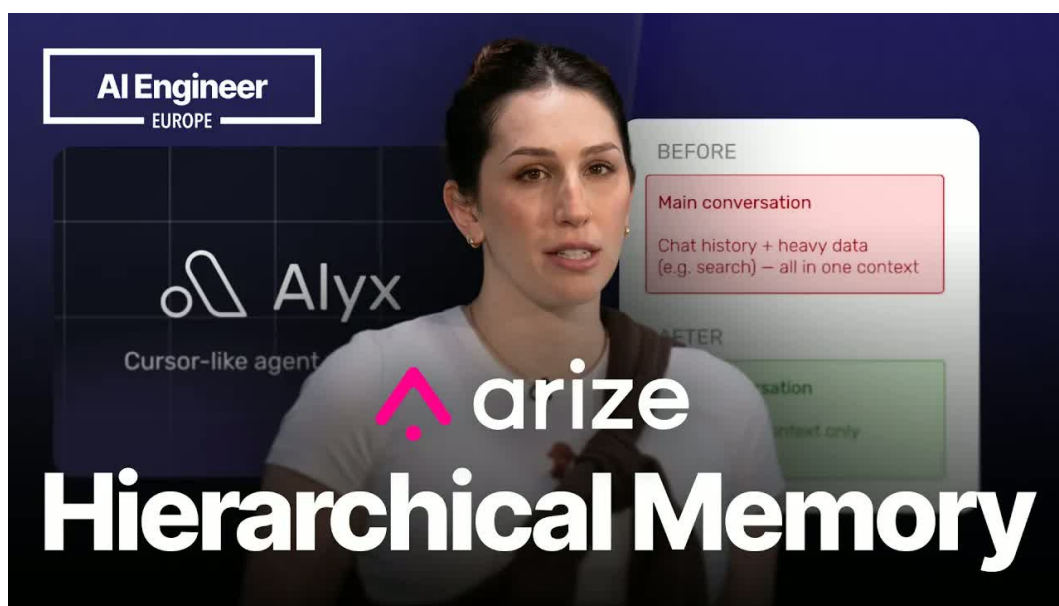


AI Engineer 演讲笔记

我们如何解决智能体中的上下文管理问题

KrillinAI 小林 / Codex 整理

视频发布：2026-05-18 时长：16:16



视频来源： <https://www.bilibili.com/video/BV1DBLc6FEe5/>

主题：上下文工程、智能截断、记忆存储、长会话评估、子代理架构与长期记忆。

素材：B 站平台 AI 中文字幕、1080p 本地视频帧、演讲原始封面与重绘教学图。

目录

1 上下文工程的战场：模型到底看见什么	3
1.1 Alex/Rise 案例：上下文问题从哪里来	3
1.2 从“塞满窗口”到“选择证据”	3
1.3 为什么这是产品问题	5
1.4 本章小结	5
2 恶性循环：代理分析自己的数据时为什么会失控	5
2.1 用 Alex 构建 Alex：闭环从这里形成	5
2.2 从跨度增长到上下文触顶	6
2.3 分析系统受制于被分析数据	7
2.4 本章小结	7
3 从粗暴截断到智能截断记忆	7
3.1 第一种尝试：粗暴截断为什么会让追问失忆	7
3.2 第二种尝试：摘要压缩为什么把控制权交给了模型	8
3.3 第三种尝试：智能截断与记忆存储的组合	10
3.4 哪些内容应该保留，哪些内容应该入库	11
3.5 为什么这套方案比前两种更稳	12
3.6 本章小结	12
4 长会话评估：让遗忘变成可测试问题	12
4.1 长会话为什么是产品里的常态	13
4.2 智能截断上线后，失败为什么移动到了更后面	13
4.3 EBO 与第 N+1 轮测试	14
4.4 上下文状态：测试真正要看的对象	14
4.5 为什么测试还不够：单个代理仍会被数据量压垮	15
4.6 本章小结	15
5 子代理架构：把重数据留在局部上下文	15
5.1 主代理、子代理与任务委托	15
5.2 重数据任务为什么会污染主上下文	16
5.3 上下文分区：把臃肿数据锁在局部上下文	16

5.4	子代理怎样让主上下文保持精简	18
5.5	架构收益与边界	18
5.6	通往下一章：仍然存在的长期问题	19
5.7	本章小结	19
6	未解问题与最终综合：长期记忆、启发式预算和上下文优先	19
6.1	总结失效以后，真正暴露出来的问题	19
6.2	长期记忆：从本轮可回溯到跨会话可延续	20
6.3	上下文预算：先分配窗口，再填充内容	21
6.4	上下文质量评估：别只改提示词，要测上下文	22
6.5	Claude Code 线索与问答中的实现细节	22
6.6	本章小结	23

1 上下文工程的战场：模型到底看见什么

1.1 Alex/Rise 案例：上下文问题从哪里来

演讲者在 00:01:08–00:01:24 先把 Alex 放到一个具体产品场景里：Alex 是一个帮助开发者构建 AI 应用的 AI 框架，内置提示优化、数据增强、标注管理等四十多项能力。这个开场的价值不在于介绍功能清单，而在于说明后面的上下文问题有真实的数据来源。Alex 位于 Rise 平台之上，而 Rise 是可观测性平台；因此 Alex 需要读取用户提示、交互元数据、追踪数据、跨度以及工具运行结果。

可观测性里的“飞行记录仪”可以把追踪数据理解为一次请求或一次代理工作流的飞行记录仪。追踪记录描述“这次任务从哪里开始、经过哪些步骤、哪些工具被调用、哪里耗时或报错”；跨度则是追踪记录里的一个具体片段，例如一次工具调用、一次检索、一次模型回答或一次用户交互。单个跨度 s_i 看起来很小，几十个、几百个跨度连在一起，就会变成模型需要筛选的大型证据堆。

本章先锁定几个后续章节会反复使用的术语。AI 代理指由大语言模型驱动、能够推理、调用工具并延续工作流的系统；Alex 是本讲的案例系统。上下文窗口是模型本轮可直接读取的有限输入空间，可以把它想成模型面前的工作台。token 预算是这张工作台可容纳的最大输入与输出配额。上下文工程则是围绕这张工作台做的系统设计：给定大量候选材料，决定哪些内容进入模型眼前，哪些内容暂时留在外部。

上下文工程的定义 上下文工程是对“模型当前看见什么”的战略控制。它处理的核心变量有三个：候选数据 D 、token 预算 B 、最终进入上下文窗口的实时上下文 C 。提示词写得再漂亮，如果 C 里缺少关键证据，AI 代理仍然会给出错误答案。

1.2 从“塞满窗口”到“选择证据”

00:01:48–00:02:11 这段话给出全讲的转折：团队逐渐意识到，上下文才是决定代理成败的关键。这里的“成败”非常工程化，并非抽象口号。到 00:02:33–00:02:50，演讲者明确说，上下文工程是战略性选择模型看到的内容；问题不只是“还剩 x 个 token，尽可能塞进去”，更要判断哪些数据对当前任务最关键。

这个判断可以形式化为一个选择函数：

$$C = \text{Select}(D, B), \quad |C|_{\text{tok}} \leq B$$

其中：

- D ：可用候选材料，包括提示、历史消息、工具结果、追踪数据、跨度和元数据。
- B ：token 预算，即本轮模型调用可承载的上下文上限。
- C ：实际放入上下文窗口的实时上下文。

- $\text{Select}(D, B)$: 上下文选择策略, 负责在预算限制下挑选材料。
- $|C|_{\text{tok}}$: 实时上下文按 token 计量后的长度。

这个公式的直觉很朴素: D 像仓库, C 像工作台, B 像工作台面积。真正困难的地方在于, 工作台面积固定, 仓库里的证据还会随着用户会话、工具调用和系统日志不断增加。把所有东西都堆上来会挤爆窗口; 随手丢掉材料又可能丢掉关键线索。

Why Context Management Matters

Context engineering = choosing what the model sees



图 1: 讲者给出核心定义: 上下文工程就是选择模型能看到什么。视频源 `sources/video/source.mp4`, 00:02:28–00:02:43。

Alyx Reality

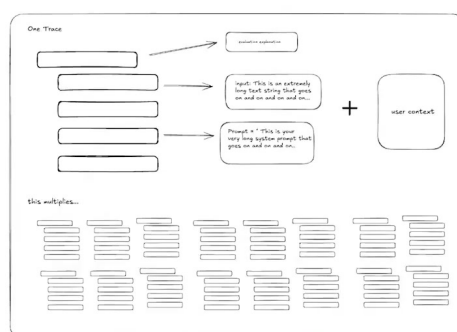


图 2: Alex/Rise 的真实数据面貌: 一次追踪数据会展开为提示、跨度、用户上下文和大量派生记录。视频源 `sources/video/source.mp4`, 00:03:00–00:03:25。

常见误解: 上下文窗口越满越好 “填满 token 预算” 只能保证模型读到更多字符, 不能保证模型读到正确证据。对于 Alex 这类产品, 错误的可见数据会直接变成错误回答; 错误回答会伤害用户信任, 最终成为产品体验问题。

1.3 为什么这是产品问题

00:03:00–00:03:25 是第一个具体压力点。Alex 建立在 Rise 平台之上，需要处理 AI 代理产生的追踪数据；一条追踪记录里包括用户输入的提示内容、用户与 Alex 交互的元数据，以及多个跨度。当团队只分析单条追踪时，数据已经会变大；当任务变成“分析所有追踪记录里的模式”时，候选数据会持续增长。

这也解释了为什么演讲者在 00:03:44–00:04:03 把上下文管理提升为产品问题。工程师当然可以实现截断、摘要、检索、缓存等策略；但用户只会看到结果：Alex 是否拿到了正确数据，是否理解用户刚才问的内容，是否能在复杂追踪里找到真正相关的跨度。若上下文选择失败，模型表面上仍然在回答，实际是在错误证据上推理。

第一节的核心判断 上下文窗口控制的是“模型此刻能看见的世界”。记忆像资料室，能保存和取回材料；上下文选择像把资料搬到桌面的动作。可靠 AI 代理的第一道关卡，就是在有限 token 预算下把正确证据搬到模型眼前。

1.4 本章小结

本节完成了后续章节的概念地基：AI 代理是能够推理、调用工具并延续工作流的系统；Alex 是运行在 Rise 平台上的案例系统；追踪数据记录代理、用户和工具的执行证据；跨度是追踪中的单个操作片段；元数据补充说明交互与追踪的属性。上下文工程的核心任务，是在候选数据 D 和 token 预算 B 之间执行上下文选择，得到模型真正可见的实时上下文 C 。下一节会看到，当 Alex 分析的追踪数据也由 Alex 自己制造时，这个选择问题会变成自我放大的失控循环。

2 恶性循环：代理分析自己的数据时为什么会失控

2.1 用 Alex 构建 Alex：闭环从这里形成

00:04:06–00:04:21，演讲者开始描述团队遇到的恶性循环。团队的产品判断很直接：如果能构建一个让 AI 应用开发更轻松的代理，用户就会真正需要它。于是他们用 Alex 构建 Alex。这个选择在产品上合理，在数据系统上却立刻引入一个闭环：Alex 的工作会产生追踪数据和跨度；Alex 又要读取这些追踪数据和跨度来分析、调试、改进自己。

本节把恶性循环定义为：系统为了完成任务而生成更多证据，更多证据推高上下文负载，负载触顶引发失败，失败和重试继续生成证据。它像一条把烟雾报警器接到造烟机上的链路；报警越多，系统越需要分析日志，日志越多，分析越容易触顶。

恶性循环的定义 恶性循环是由代理生成数据、再分析自身数据所形成的自我放大失败链。它的关键并不在单次失败，而在失败会留下新的追踪和跨度，使下一次运行要处理的候选数据更大。

2.2 从跨度增长到上下文触顶

00:04:21–00:04:38 给出了闭环的逐步机制：Alex 在追踪和跨度数据上运行；跨度数据不断增长；数据量过多后达到上下文限制；Alex 失败；跨度数据保留失败信息；团队再次尝试，并向其中添加更多数据；再次运行后又失败。这里的技术词“上下文膨胀”指候选数据规模 $|D_t|$ 随时间或重试轮次增加，逐渐超过可用上下文窗口。

可以用一个递推式表达这个过程：

$$D_{t+1} = D_t + \Delta_{\text{trace}}(\text{failure}_t, \text{retry}_t), \quad |D_{t+1}|_{\text{tok}} > |D_t|_{\text{tok}}$$

其中：

- D_t ：第 t 次运行前已有的候选数据。
- D_{t+1} ：第 $t+1$ 次运行前需要面对的候选数据。
- $\Delta_{\text{trace}}(\text{failure}_t, \text{retry}_t)$ ：第 t 次失败与重试新增的追踪、跨度、错误信息和交互记录。
- $|D_t|_{\text{tok}}$ ：候选数据按 token 计量后的规模。
- B ：上一节定义的 token 预算；当 $|D_t|_{\text{tok}}$ 持续逼近或超过 B ，上下文选择策略必须丢弃、压缩或外移材料。

这个式子真正危险的地方在于 Δ_{trace} 通常为正。每一次失败都会留下更多证据；每一次重试又会读取更多旧证据和新证据。若选择策略没有变化，重试不一定带来恢复，反而会把下一轮推向更高的数据水位。

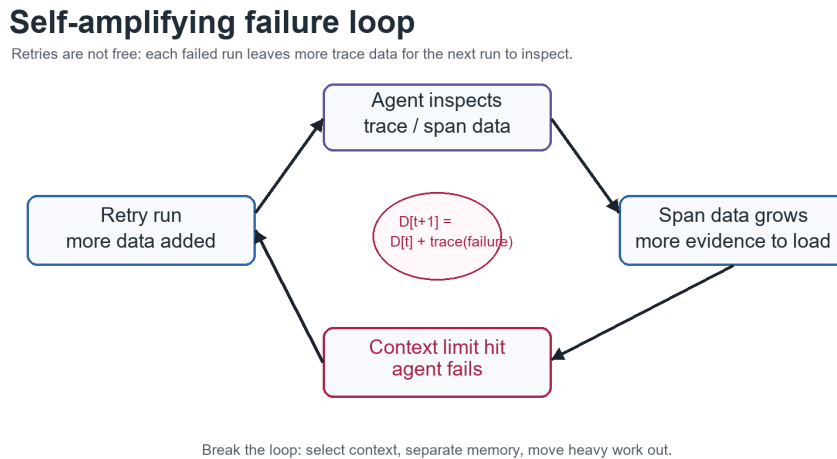


图 3：教学重绘的失败循环：重试会把失败留下的追踪数据继续加入未来输入，数据规模可写作 $D_{t+1} = D_t + \Delta_{\text{trace}}(\text{failure}_t)$ 。

重试日志并不免费 在普通后端服务里，重试常被看成恢复手段；在数据密集型 AI 代理里，重试还会制造下一轮要读的材料。若失败原因是上下文触顶，盲目重试可能让 D_t 继续增长，使后续选择更难。

2.3 分析系统受制于被分析数据

00:04:42–00:04:48，演讲者把根因说得很硬：分析数据的系统受制于数据本身。换成系统语言，就是分析器 A 的输入空间受到 B 限制，而被分析对象 D_{trace} 会随着系统运行不断扩展。当 D_{trace} 的关键部分无法完整进入 C ，Alex 就会出现两类失败：一类是遗漏关键跨度，导致诊断方向错误；另一类是保留了太多噪声，导致模型在无关细节里耗尽注意力。

因此，任务失败-重试循环不能简单归结为模型能力问题。它是输入选择、可观测性数据和产品工作流共同制造的问题。模型可以很强，但它不能推理自己没有看到的证据，也不能稳定忽略被硬塞进窗口的大量噪声。到 00:04:53–00:05:09，团队给出了后续三步方案：真正控制上下文，将上下文与记忆分离，把繁重任务转移到其他代理。这三步分别会在后续章节展开。

为什么“上下文”和“记忆”要分开讨论 上下文是模型本轮直接可见的材料，决定“此刻怎么推理”；记忆是上下文窗口外的可恢复材料，决定“以后还能不能找回”。恶性循环出现时，若把所有记忆都当成实时上下文塞给模型，系统会更快触顶；若把所有旧材料都丢掉，系统又会失去诊断依据。

2.4 本章小结

本节定义了恶性循环、上下文膨胀和任务失败-重试循环。Alex 团队用 Alex 构建 Alex，使代理既生成追踪和跨度，又读取这些数据来分析自身行为；当跨度增长推高候选数据规模，固定 token 预算无法容纳全部材料，失败会发生；失败留下新的追踪信息，重试继续增加负载。下一章的关键问题因此变得清楚：团队必须控制实时上下文，并把可恢复的历史材料放到记忆中，才能打断这个自我放大的循环。

3 从粗暴截断到智能截断记忆

上一章已经把压力来源讲清楚了：Alex 在分析自身产生的追踪数据时，会被越来越长的对话、跨度、工具调用结果和重试记录推向上下文窗口上限。此时继续说“把更多材料塞进去”没有意义，因为 token 预算像一张有限桌面，桌面满了以后，真正的问题变成：哪些材料必须留在桌上，哪些材料可以先放进抽屉，等需要时再拿回来。

本章讨论团队在这条路上的三次尝试：先是粗暴截断，然后是摘要压缩，最后形成智能截断加记忆存储的组合。三者的真正差别超出“短一点”这个表面动作，关键在于谁拥有证据选择权：固定规则、LLM 自行概括，还是由系统策略保护关键位置并提供可回溯上下文。

3.1 第一种尝试：粗暴截断为什么会让追问失忆

粗暴截断指最直接的丢弃策略：当上下文太长时，只保留开头一段，例如最初的一百个字符，把剩余内容直接舍弃。这个做法看起来很有诱惑力，因为开头通常包含任务目标、用户最初的问题、系统意图或背景设定；在非常简单的单轮任务里，它甚至会显得“够用”。

问题出现在追问场景。用户先问“最常见的输入是什么”，Alex 给出若干输入后，用户继续问

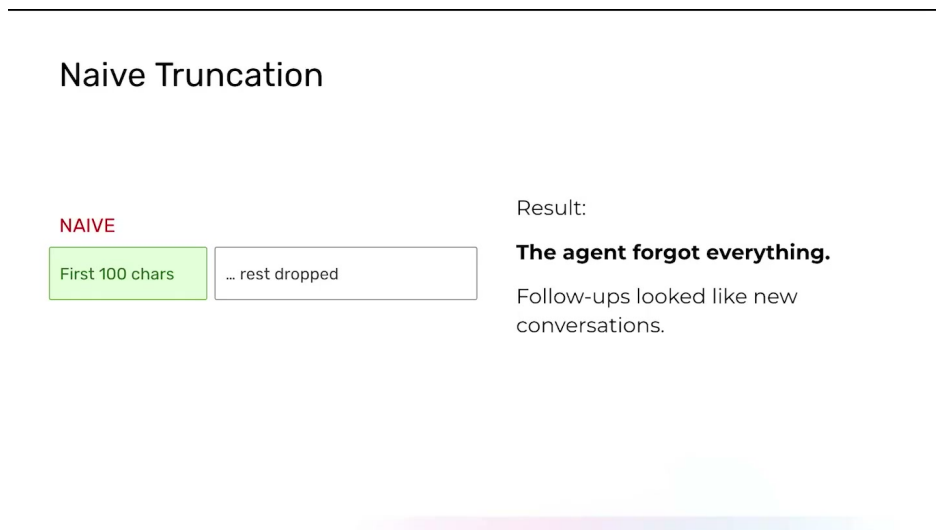


图 4: 粗暴截断只保留开头片段, 后续追问会失去已建立的引用关系; 图中 “First 100 chars” 与 “rest dropped” 把这种信息断裂直观化。来源区间: 00:05:31–00:05:47。

“能多说一点关于输入 B 的信息吗”。如果粗暴截断只留下最早的上下文, 刚刚出现的 “输入 B” 很可能已经被剪掉。模型看到的材料里没有指代对象, 后续问题就像一段缺页小说: 读者知道有人在问 “它”, 却不知道 “它” 指谁。

粗暴截断的真实损伤 粗暴截断省下了 token, 却破坏了推理链。它常常丢失最近形成的引用关系、工具结果和中间判断, 这些内容远比表面的背景噪声更致命。对 AI 代理来说, 指代关系一断, 追问就会退化成新对话; 模型并没有真正变笨, 只是被拿走了完成推理所需的证据。

从形式上看, 粗暴截断相当于把可见上下文近似成 $C_{\text{live}} \approx H_k$, 其中 H_k 是保留下来的头部片段。这个近似隐含了一个危险假设: 头部永远比尾部和中间更重要。真实对话并不服从这个假设。任务目标常在头部, 最新工具结果常在尾部, 关键证据可能埋在中间; 只保留头部, 就像只看病历第一页便开药, 速度很快, 误判也很快。

3.2 第二种尝试: 摘要压缩为什么把控制权交给了模型

粗暴截断失败后, 一个自然想法是摘要压缩: 既然 LLM 擅长总结, 那就让它把长上下文压缩到更短的 token 数, 再把摘要交给后续模型。这个方案听起来很顺, 因为它似乎同时保留了全局信息和 token 预算。

但团队很快发现, 摘要压缩的稳定性不足。核心原因是: 摘要并非中性的缩小镜, 它会重排证据、合并细节、删除模型当下认为次要的内容。对写文章来说, 这种概括能力很有用; 对调试代理、追踪工具调用、解释某次失败来说, 被删掉的细节可能正是关键证据。

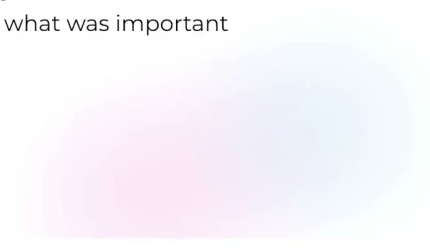
摘要压缩的控制权问题 摘要压缩把 “什么重要” 这件事交给 LLM 自行判断。系统需要的是可审计的上下文选择策略, 摘要却常常变成一次不可控的证据改写: 哪些工具调用被保留、哪些错误被省略、哪些用户限定条件被淡化, 都可能取决于模型在那一刻的概括偏好。

LLM summarization as compression

Sounded like the obvious solution.

But:

- too inconsistent
- No control over what was important
- unreliable



arize

We Make AI Work

© All Rights Reserved

图 5: 把长上下文交给 LLM 摘要压缩会引入选择权漂移: 重要性判断被摘要器接管, 输出表现为不稳定、不可控和不可靠。来源区间: 00:06:16–00:06:41。

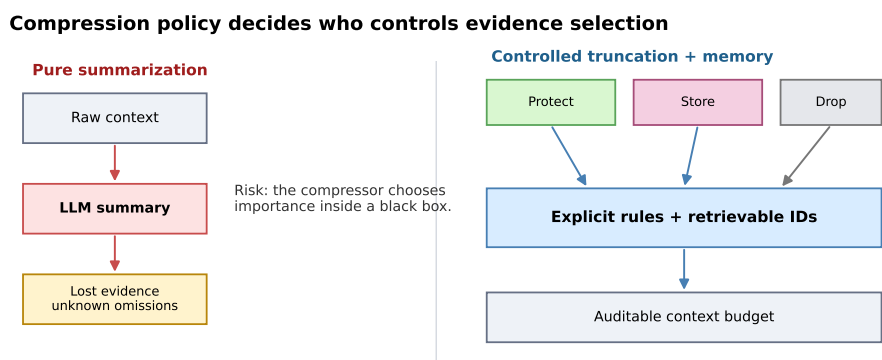


图 6: 摘要压缩与受控截断的关键差异在于证据选择权: 前者把取舍交给黑箱摘要, 后者用显式规则把内容分为保护、存储和丢弃三类。

可以把摘要压缩写成：

$$S = \text{Summarize}(D)$$

其中 D 是候选材料， S 是压缩后的摘要。这个表达式看似干净，真正麻烦在于 Summarize 本身不透明。若摘要函数删除了一个失败跨度 s_i ，后续模型就无法知道这条证据曾经存在；若摘要把用户的限定条件改写得过宽，后续回答就会在错误边界内显得很自信。

“压缩”和“检索”的关键差异 压缩会把原始材料改写成更短文本，信息一旦被丢掉，后续步骤很难恢复；检索则把原始材料留在记忆存储中，只在需要时取回相关片段。前者像把一摞发票揉成一段报销说明，后者像给发票编号归档。说明方便阅读，归档方便查证。

3.3 第三种尝试：智能截断与记忆存储的组合

Alex 最终采用的是智能截断加记忆存储。它的做法可以概括为：保留头部，保留尾部，把中间部分存入可检索的记忆；同时保护系统提示和最新工具结果，避免它们被普通历史消息挤出上下文窗口。

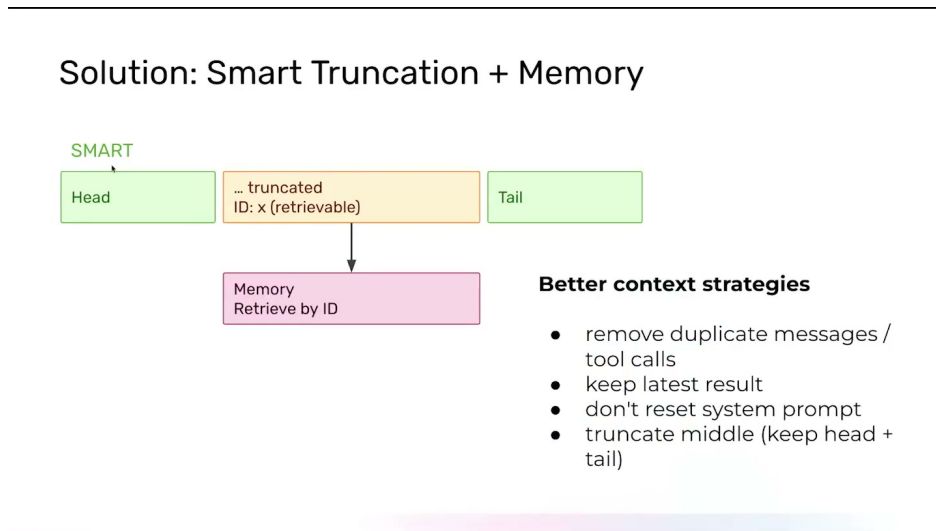


图 7：智能截断加记忆把开头与结尾保留在实时上下文中，将中间内容以可检索 ID 放入记忆，并显式保护最新工具结果和系统提示。来源区间：00:06:44–00:07:16。

这套设计里，头尾保留承担两个不同职责。头部 H_k 像任务的“宪章”：它通常包含系统提示、最初目标、用户设定和重要约束，帮助代理保持方向。尾部 T_k 像工作台上的“最新实验记录”：它包含最近几轮对话、最新工具调用结果、刚刚形成的指代关系和用户的补充要求。中间材料不直接丢弃，会进入记忆存储 M ，成为之后可取回的可回溯上下文。

本章的核心判断 上下文决定模型此刻能看见什么，记忆决定哪些内容以后仍然能被找回。智能截断的价值在于把“当前可见”和“未来可回溯”分开管理：活跃推理依赖头尾，证据恢复依赖记忆存储。

这可以写成一个简洁的形式化表达：

$$C_{\text{live}} = H_k \cup T_k \cup R(q, M)$$

Live context = protected head + recent tail + retrieved memory

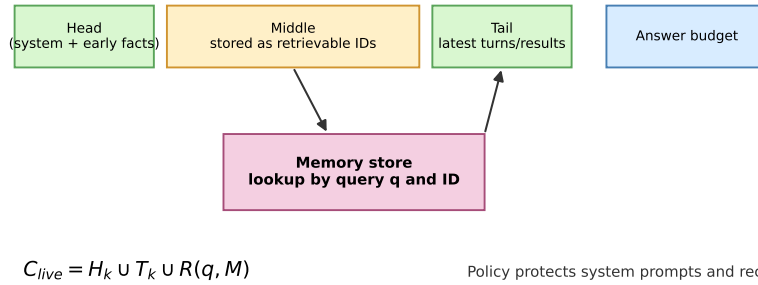


图 8: 头尾保留与记忆检索可以写成 $C_{live} = H_k \cup T_k \cup R(q, M)$: 头部 H_k 承载系统约束和早期事实, 尾部 T_k 承载最新状态, 检索函数 $R(q, M)$ 从记忆 M 中取回与当前查询 q 相关的中间证据。

符号解释如下:

- C_{live} : 当前送入模型的实时上下文。
- H_k : 被保留的头部片段, 通常包含系统提示、初始目标和全局约束。
- T_k : 被保留的尾部片段, 通常包含最近对话、最新工具结果和新形成的引用关系。
- M : 记忆存储, 保存被移出实时上下文的中间材料、较旧工具调用载荷和重复消息。
- q : 当前问题或当前推理需求, 用来触发相关记忆检索。
- $R(q, M)$: 从记忆存储中按 q 检索出的相关片段, 也就是可回溯上下文。

这个公式里的并集 $\cup$ 不必理解成数学集合的去重拼接; 在工程上, 它表示上下文装配: 先放入受保护的头部和尾部, 再按当前问题从记忆存储中取回相关证据。真实实现还会受 token 预算限制, 因此 $R(q, M)$ 不能无限取回。系统仍要排序、裁剪、去重, 只是这些动作发生在可控策略下, 而非一次性让摘要模型决定全部证据命运。

3.4 哪些内容应该保留, 哪些内容应该入库

智能截断的实际难点在于分类。只说“保留重要内容”没有工程价值, 因为重要性要落到规则上。根据本段案例, 可以把上下文材料分成三类。

- 必须保护的材料: 系统提示、初始任务目标、最新用户问题、最新工具调用结果, 以及最近几轮形成的指代关系。
- 适合入库的材料: 冗长的中间对话、重复消息、较旧工具调用载荷、已经用过但未来可能查证的追踪数据片段。
- 需要按需回溯的材料: 用户前文提到的对象、较早工具结果中的具体字段、某次失败跨度、以及解释当前问题所需的历史判断。

这里有一个容易被低估的细节: 工具调用往往比普通聊天更占 token。一次日志搜索、一次

trace 查询、一次 span 展开，都可能返回大量结构化文本。若这些载荷长期留在实时上下文里，它们会像沉重附件一样拖慢每一轮推理；若直接丢弃，代理在后续追问时又无法查证。把它们放入记忆存储，并保留最新结果，是一种更可控的折中。

工程直觉：头尾保留像“书签 + 当前页”头部像书签，告诉读者这本书为何而读、读到哪里不能偏题；尾部像当前页，承载最近刚出现的人名、变量和结论；记忆存储像索引，允许读者在需要时回翻中间章节。智能截断把这三者组合起来，避免把整本书摊满桌面。

3.5 为什么这套方案比前两种更稳

智能截断更稳，原因并非它神奇地增加了上下文窗口；它真正降低的是错误丢弃关键证据的概率。粗暴截断只有一个方向：砍掉剩余内容。摘要压缩只有一个黑箱：让模型概括。智能截断则拆成三个较小的问题：保护哪里、存储哪里、检索哪里。

第一，系统提示和最新工具结果被保护，代理不会在关键规则和最新事实上“断电”。第二，中间部分进入记忆存储，历史材料有了恢复路径。第三，可回溯上下文由当前问题 q 触发，意味着不同追问可以取回不同证据；用户追问“输入 B”时，系统可以尝试找回包含输入 B 的旧片段，而不必指望摘要恰好保留了它。

从上下文管理到证据管理 这一章的真正转向是：上下文管理除了缩短文本，还要管理证据的生命周期。证据可以当前可见、暂时入库、按需回溯，也可能因过期或重复被降级。成熟的 AI 代理需要这样的生命周期设计，否则每一次截断都像一次失忆手术。

当然，这套方案也有边界。它在几个月里给团队带来了稳定效果，但长时间会话继续增长时，新的退化会在更晚的轮次出现。换句话说，智能截断解决了“马上爆窗”的问题，却没有自动解决“二十轮以后还记得准不准”的问题。下一章因此要进入长会话评估：把遗忘从用户投诉变成可测试、可复现、可提前发现的问题。

3.6 本章小结

本章用三次尝试解释了 Alex 的上下文控制路径。粗暴截断保留头部、丢掉其余内容，在简单任务中看似有效，在追问中会切断引用关系。摘要压缩把长材料改写成短摘要，节省 token 的同时让 LLM 获得过多证据选择权，稳定性不足。智能截断通过头尾保留、记忆存储和可回溯上下文，把实时可见材料与未来可查材料分开管理。

读者需要带走的判断很直接：有限 token 预算下，好的代理系统不能只问“删掉多少”，还要问“删掉后能否找回、找回时是否可信、最新事实是否被保护”。这就是从粗暴截断走向智能截断记忆的工程分水岭。

4 长会话评估：让遗忘变成可测试问题

上一章给出了一个阶段性稳定方案：智能截断保护头尾，把中间材料放入记忆存储，并在需要时取回可回溯上下文。这个方案确实缓解了上下文窗口被快速塞满的问题，但它也把故障形态

改了：失败不再总是很早暴露，许多问题会被推迟到对话后期才出现。工程上，这更危险，因为系统前几轮看起来正常，用户也会因此把更多任务、更多背景、更多追问继续推进同一个会话。

本章的重点是长会话评估。它把“代理会不会在后面忘掉东西”从用户投诉变成可以提前构造、重复运行、观察上下文状态的测试问题。换句话说，团队开始把上下文质量纳入评估对象，而不只是在运行时做清理。

4.1 长会话为什么是产品里的常态

长会话指用户把一个对话或工作流持续推进很多轮，用户通常不会为每个小问题重新开一个会话。对使用 Claude、Cursor 或 Alex 这类工作流助手的人来说，这种行为很自然：用户已经在同一个会话里解释了项目、目标、错误现象和约束，就会期待后续追问沿着同一条线继续下去。

这和普通问答产品的单轮测试差异很大。单轮测试像检查一张答题卡：题目、材料和答案都在同一页。长会话更像接力赛：第 2 轮形成的决定、第 5 轮出现的工具结果、第 8 轮补充的限制条件，可能都要影响第 11 轮的回答。只要其中某个关键事实在上下文状态里消失，模型就会在表面流畅的语言中做出错误判断。

长会话的隐藏难点 长会话里的“记住”并不等于把所有历史原封不动塞进上下文窗口。真正要保证的是：当第 t 轮需要某个早期事实时，系统能让这个事实以可信、相关、足够具体的方式回到模型眼前。这里考验的是上下文选择、记忆检索和评估设计的组合能力。

4.2 智能截断上线后，失败为什么移动到了更后面

团队首次实施智能截断时，早期效果看起来不错。原因也直观：系统提示、最新工具结果、头部目标和尾部对话被保护后，Alex 不容易在短时间内撞上上下文限制；那些冗长的中间材料也不会直接挤爆实时上下文。

但随着对话越来越长，新的退化开始出现。智能截断把“马上失败”的概率降了下来，却没有保证二十轮以后仍能保留所有必要事实。对话增长到一定程度后，关键事实可能落入中间历史，记忆检索可能没有把它召回，或者召回内容太粗，导致模型只看到一个模糊线索。最终表现就是：Alex 在对话后期逐渐遗忘信息，直到用户报告问题，或者团队回看数据时才发现。

更晚暴露的故障更难抓 早期爆掉的上下文问题很容易定位：日志里通常能看到窗口溢出、工具结果缺失或明显截断。后期遗忘更麻烦，因为模型仍然能回答，只是答案悄悄偏离了早期事实。系统看起来在工作，实际上已经把某些约束、指代或结论丢在了身后。

可以把这种现象理解成“故障时间后移”。智能截断延长了系统可用时间，就像给车辆换了更大的油箱；但如果没有里程表和定期检查，驾驶者仍然可能在更远处耗尽燃料。长会话评估要做的事，就是给这种后期退化建立仪表盘。

4.3 EBO 与第 N+1 轮测试

演讲者提到团队引入了长会话 EBO。这里必须谨慎：字幕只给出“EBOs”，没有展开这个缩写。为了不把未证实的信息写死，本讲义把 EBO 当作一种评估用例或 expected behavior objective，即“期望行为目标”。它的作用是描述系统在某类长会话场景中应该保持的行为，同时避免给缩写编造一个确定来源。

团队采用的具体方法可以概括为第 N+1 轮测试：先加载 N 轮历史对话，例如 10 轮，再测试第 N+1 轮，例如第 11 轮，用这一轮探针观察上下文状态 S_{N+1} 。如果第 11 轮必须引用第 3 轮的用户偏好、第 6 轮的工具结果和第 9 轮的最新约束，那么测试就能直接检查这些事实是否仍然可用。

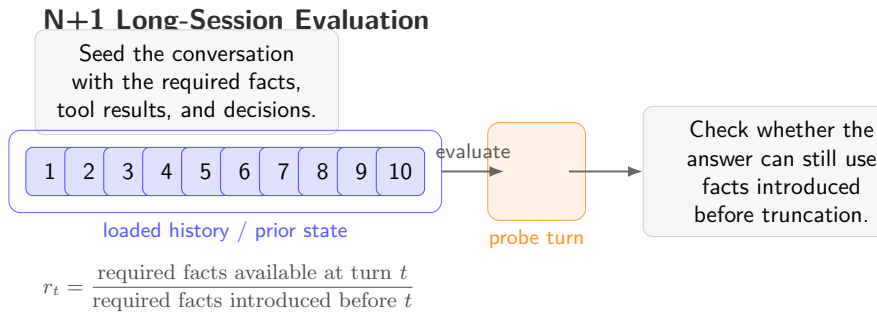


图 9: $N+1$ 轮评估示意：前 N 轮构造历史状态，第 $N+1$ 轮检查关键事实是否仍在可用上下文中。

第 $N+1$ 轮测试的核心 第 $N+1$ 轮测试把“它以后会不会忘”变成“给定前 N 轮历史，第 $N+1$ 轮能否取回并正确使用必要事实”。这类测试不依赖用户先踩坑，可以在版本发布前反复运行，用来比较不同截断策略、记忆检索策略和上下文装配策略。

一个朴素指标是保留率：

$$r_t = \frac{\text{第 } t \text{ 轮可用的必要事实数}}{\text{第 } t \text{ 轮之前引入的必要事实数}}$$

这个公式只表达直觉，不要求把所有事实都机械计数。分子表示在第 t 轮真正能被模型使用的关键信息，分母表示按测试设计应该被保留的关键信息。若 r_{11} 下降，就说明前 10 轮里某些必要事实没有进入第 11 轮的上下文状态。对工程团队来说，它至少提供了一个讨论抓手：到底是截断策略删错了，记忆存储没有召回，还是召回内容进入窗口时被其他材料挤掉了。

4.4 上下文状态：测试真正要看的对象

上下文状态 S_t 指第 t 轮时系统实际保有、可见或可取回的信息状态。它比“对话历史长度”更重要。历史很长并不代表模型拥有正确上下文；历史很短也不必然表示信息不足。关键问题是：当前任务需要的事实、约束、工具结果和用户意图，是否处在可被模型使用的位置上。

在长会话测试里， S_t 至少应检查四类内容。第一，早期目标是否仍然可见或可回溯。第二，最近工具调用结果是否被保护。第三，中间轮次的关键决策是否能从记忆存储找回。第四，模型是

否把不同轮次的事实正确组合，避免只回忆到孤立片段。

从“测回答”到“测状态”只看最终回答，团队只能知道结果错了；观察上下文状态，团队可以知道为什么错。错误可能来自事实没有被保存、保存了却没有召回、召回了却没有放进实时上下文，或者放进去了但被噪声淹没。长会话评估的价值正在这里：它让故障链条可以分段检查。

4.5 为什么测试还不够：单个代理仍会被数据量压垮

长会话 EBO 和第 $N+1$ 轮测试解决的是可测试性问题，它们能提前暴露遗忘，却不能自动减少数据量。演讲者在这里顺势引出下一步结论：有些场景下，数据量对单个代理来说仍然过大。尤其是 Alex 需要搜索追踪数据、展开跨度、查看追踪栈、执行多个查询时，单个主对话同时承载聊天历史、搜索过程和中间推理，会迅速变得臃肿。

这就是第 5 节的入口。评估告诉团队“什么时候忘了”，架构要回答“怎样减少主上下文承担的重量”。下一章要讲的子代理架构，正是把重数据任务从主对话里拆出去，让主代理保持轻量。

4.6 本章小结

本章说明了智能截断之后出现的新问题：故障不一定立刻发生，常常被推迟到更长对话的后期。用户倾向于保持单一会话，尤其在代码、调试和数据分析工作流里，早期事实会持续影响后续决策；一旦这些事实从上下文状态里消失，模型就会在看似自然的回答中犯错。

团队用长会话 EBO 和第 $N+1$ 轮测试把这种后期遗忘变成可测试问题：加载前 N 轮历史，在第 $N+1$ 轮检查必要事实是否仍然可用。保留率 r_t 可以作为直观指标，帮助定位截断、检索和上下文装配环节的损伤。不过，测试只能暴露问题，不能替代代理架构分担数据压力。面对追踪搜索和大量跨度，系统还需要子代理来隔离重数据任务。

5 子代理架构：把重数据留在局部上下文

上一节把遗忘变成了可测试问题；这一节处理另一个更偏系统设计的问题：有些材料从一开始就不该全部进入主对话。Alex 面对的追踪搜索可能包含数百个跨度、多个查询、成串工具调用结果和大量中间推理。如果这些内容与用户聊天历史混在同一个上下文窗口里，主代理会越来越像一张堆满日志、便签、草稿和旧报表的办公桌，任何下一步行动都要在噪声里翻找。

子代理架构的核心思路是上下文分区：主代理保留用户可见的主线、目标和必要上下文；子代理接收被委托的重数据任务，在自己的局部上下文里处理大量追踪数据，然后把紧凑结果返回给主代理。这样，主对话继续轻量推进，重数据留在局部上下文中消化。

5.1 主代理、子代理与任务委托

主代理是面向用户的代理，记为 A_{main} 。它负责理解用户目标、维护对话主线、决定下一步行动，并把最终结果解释给用户。子代理是被委托的工作代理，记为 A_{sub} 。它可以拥有自己的局部

上下文 C_{sub} ，专门处理某个边界清楚的任务，例如搜索追踪数据、检查一组跨度、比较多个工具调用结果。

任务委托可以写成：

$$A_{\text{main}} \rightarrow A_{\text{sub}} : \text{任务说明} + \text{必要约束}$$

随后子代理返回：

$$A_{\text{sub}} \rightarrow A_{\text{main}} : y_{\text{sub}}$$

其中 y_{sub} 是紧凑结果。它不应包含所有原始日志和完整推理草稿，而应包含主代理继续工作所需的结论、证据索引、关键异常、置信边界和必要引用。

子代理的接口纪律 子代理的价值来自边界清楚的任务委托。主代理交给它一个可执行的小问题，子代理在局部上下文里处理大材料，再返回紧凑结果。若子代理把全部中间数据原样倒回主代理，分区收益就会消失。

5.2 重数据任务为什么会污染主上下文

重数据任务指需要大量追踪数据、跨度、搜索结果或中间推理才能完成的操作。演讲者举的例子是 Alex 在数据中搜索并生成结果：它可能要查看主流程、追踪栈，甚至单个追踪里数百个跨度；还可能反复发起多个查询，逐步判断哪些数据值得继续展开。

如果所有这些内容都进入主代理的上下文，问题会同时出现在三个层面。第一，token 预算被原始数据消耗，用户目标和最新约束被挤压。第二，中间推理大量堆积，模型会把临时假设、废弃路径和最终证据混在一起。第三，主代理每一轮都要背着这些历史继续前进，即使用户只是问一个很简单后续问题。

重数据任务的典型误伤 把追踪搜索全过程留在主对话里，等于让面向用户的代理一直携带后台检索的全部泥沙。它会降低后续回答的信噪比，也会让长会话评估中的上下文状态更难稳定。主代理需要可行动结论，无需携带每一步搜索的全部原始载荷。

5.3 上下文分区：把臃肿数据锁在局部上下文

上下文分区把主代理上下文 C_{main} 与子代理局部上下文 C_{sub} 分开。主代理只保留主对话、用户意图、关键约束和子代理返回的紧凑结果；子代理在自己的局部上下文里容纳追踪数据 D_{trace} 、跨度集合 $\{s_i\}$ 、搜索查询和中间判断。

这种结构可以用一个直观不等式表达：

$$C_{\text{main}} \ll C_{\text{main}} + C_{\text{trace}}$$

这里的 $\ll$ 表达工程直觉，并非严格数学比较：主代理承载的上下文应远小于“主对话加全部追踪材料”的总量。把 C_{trace} 留在 C_{sub} 里，主代理就不用把数百个跨度和中间搜索路径全部带进下一轮。

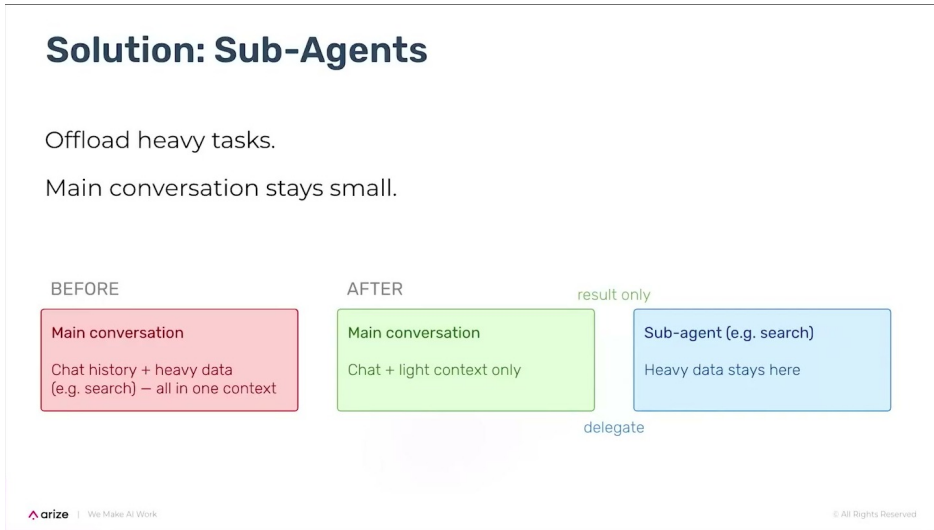


图 10: 子代理方案: 主对话只保留轻量上下文, 重数据留在局部代理里处理。视频来源区间: 00:10:18–00:10:58。

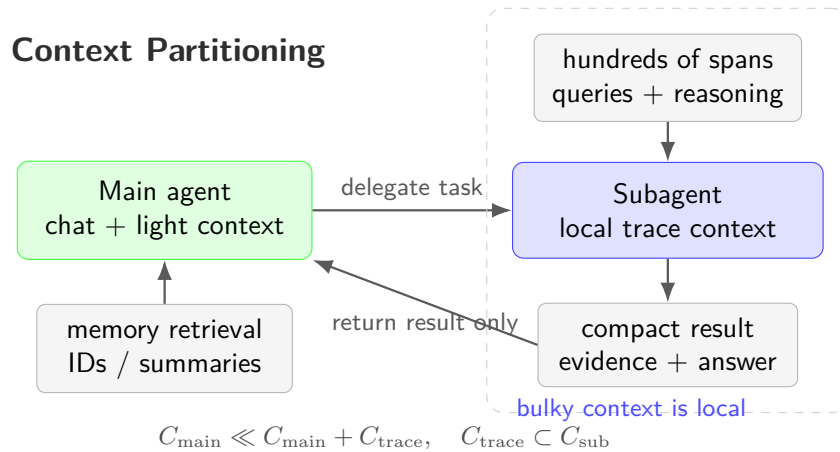


图 11: 主代理与子代理的上下文分区: C_{trace} 被限制在 C_{sub} , 主代理只接收压缩后的结果。

类比：主治医生与专科检查 主代理像主治医生，负责理解病人的整体目标和下一步治疗；子代理像专科检查室，负责读影像、化验或病理切片。主治医生需要检查结论、关键证据和风险提示，不需要把每张原始切片都摊在诊室桌上。上下文分区的直觉也是这样：局部材料在局部处理，主线决策接收压缩后的可靠结果。

5.4 子代理怎样让主上下文保持精简

子代理让主上下文精简，机制是把中间过程从主线中移出。以追踪搜索为例，主代理可以委托子代理完成如下工作：接收搜索目标，扫描相关追踪数据，展开必要跨度，过滤无关噪声，比较多个候选解释，最后返回一段紧凑结果。主代理看到的是“发现了什么、证据在哪里、下一步建议是什么”，不会被每个查询、每个失败分支、每个中间猜测占满。

这也解释了为什么子代理与记忆存储可以配合。主代理拿到紧凑结果后，用户仍然可以继续追问；如果需要更多上下文，系统可以从记忆存储或子代理产物中检索补充材料。主代理的 C_{main} 因此维持在可管理范围内，长会话继续推进时也更不容易被某一次后台检索拖垮。

为什么主代理会变轻 主代理变轻，是因为它只承载决策主线和跨轮连续性；重数据任务的原始载荷、搜索路径和临时推理被限制在子代理的局部上下文里。最终回到主代理的是紧凑结果 y_{sub} ，这让用户对话可以继续保持清晰。

5.5 架构收益与边界

演讲者把子代理称为一项非常重要的突破，因为它让数据密集型操作有了更合适的承载位置。过去，一个主对话同时包含聊天历史、复杂数据搜索和追踪栈检查；采用主代理加子代理后，主对话只保留必要上下文，繁重数据集由子代理处理，结果再传回主代理。用户仍然面对同一个连续会话，但系统内部已经把工作拆成了多个上下文空间。

不过，子代理架构也有边界。第一，任务委托必须足够清楚，否则子代理会在局部上下文里做错方向。第二，紧凑结果必须包含足够证据，否则主代理无法校验或继续推理。第三，过度拆分会增加协调成本，让系统在代理之间来回传递。第四，如果所有子代理都把完整过程回传，主上下文仍会膨胀。

子代理有边界 子代理解决的是上下文隔离和任务边界问题，不会自动提升事实质量。它需要清晰的输入合同、可靠的结果格式和必要的证据引用。否则系统只是把混乱从一个代理搬到多个代理里，调试难度还会增加。

因此，好的子代理设计要同时回答三个问题：委托什么、保留什么、回传什么。委托的是重数据任务；保留在子代理中的是局部上下文和中间推理；回传给主代理的是紧凑结果、关键证据和后续可操作建议。

5.6 通往下一章：仍然存在的长期问题

子代理架构让 Alex 能更好地处理数据密集型操作，也让主代理在长会话中保持更轻。但它没有终结上下文管理问题。超大用户输入、很长的系统提示、跨会话连续性、长期记忆和上下文预算仍然会继续施压。换句话说，子代理降低了重数据任务对主上下文的污染，却没有替代长期记忆和系统化评估。

下一章将进入这些开放问题：当用户把 workflow 拉到更长，当新会话仍要引用旧讨论，当上下文选择仍依赖启发式策略，系统还需要怎样的长期记忆、预算分配和上下文质量评估。

5.7 本章小结

本章解释了为什么数据密集型代理不能把所有材料都塞进主对话。追踪搜索可能涉及数百个跨度、多个查询和大量中间推理；这些内容若长期留在主代理上下文里，会消耗 token 预算、污染用户主线，并让长会话后期更容易退化。

子代理架构通过上下文分区解决这个问题：主代理 A_{main} 保持用户对话和决策主线，子代理 A_{sub} 在局部上下文 C_{sub} 中处理重数据任务，并向主代理返回紧凑结果 y_{sub} 。这种设计让主上下文保持精简，也为后续的长期记忆、上下文预算和上下文质量评估留下了更清晰的接口。

6 未解问题与最终综合：长期记忆、启发式预算和上下文优先

前几节已经把 Alex 的上下文治理路径讲清楚了：粗暴截断会丢线索，摘要压缩会把证据选择权过多交给模型，智能截断配合记忆存储能让主代理保持轻量，子代理又把重数据任务隔离在局部上下文里。到这里，系统已经比最初稳得多，但演讲者没有把它包装成一个完成态方案。相反，他把最后几分钟留给了更难的问题：当用户问题本身很大、系统提示很长、对话历史持续增长、代理还不断生成新的追踪数据时，平台限制仍然会被触碰。

这一节的关键提醒很直接：上下文问题不会因为换成更长的上下文窗口就自动消失。窗口变大只会推迟拥塞点；如果没有预算、记忆、评估和分区，系统迟早会把新空间也填满。

6.1 总结失效以后，真正暴露出来的问题

演讲者提到一个让团队意外的结果：摘要压缩原本看起来最自然，却没有稳定解决问题。它像把一摞工程日志交给一个人现场做读书笔记，笔记可能很流畅，但它未必保住调试所需的那一行错误、那一次工具调用、那段早期约束。相比之下，截断处理加上记忆存储的组合更有效，因为它不急着把所有信息揉成一段“听起来正确”的摘要，而是把当下要看的东西和以后还能找回的东西分开。

问题在于，分开以后仍然要选择。Alex 面对的不只是普通聊天记录，还包括系统提示、用户消息、对话历史、工具调用结果、追踪数据、跨度和元数据。客户希望 Alex 理解这些材料，可模型每次能看到的只有有限的 C_{live} 。于是团队继续使用子代理，把任务不断拆分，让不同部分处理不同上下文；这条路有效，却仍然需要持续评估是否拆得合理、是否把关键证据传回了主代理。

Context Management Roadmap

From the current working pattern toward durable agent reliability.

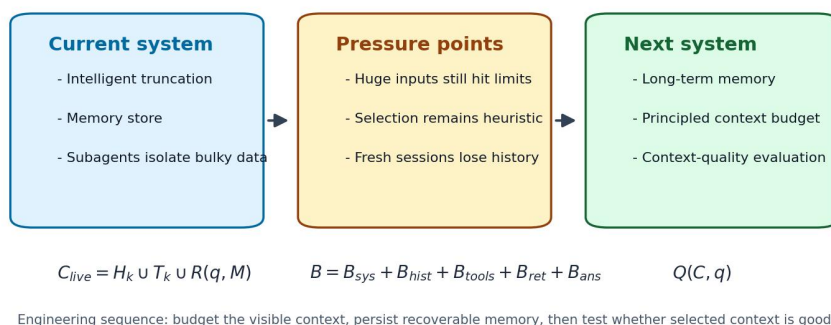


图 12: Section 6 的开放问题路线图：当前可用的智能截断、记忆存储和子代理，继续通向长期记忆、上下文预算与上下文质量评估。

本节要锁定三个开放方向：长期记忆解决跨会话连续性，上下文预算解决有限窗口的分配秩序，上下文质量评估解决“选进来的上下文是否够好”这个验证问题。三者共同决定代理能否从一次会话里的聪明，走向长期 workflows 里的可靠。

6.2 长期记忆：从本轮可回溯到跨会话可延续

用户行为正在变化。早期 Alex 的对话常常不到十轮，现在用户会把它拖到二十轮以上，跨应用执行复杂 workflow，并且在过程中不断追问。长会话让上下文状态持续承压；新会话则带来另一种断裂：用户希望引用以前与 Alex 讨论过的问题，可一旦开启新对话，Alex 天然缺少旧会话的现场。

因此，团队把长期记忆放到当前优先级很高的位置。这里需要把四个概念并排看，避免混用。

Storage and Visibility Layers

Compare lifetime, access path, owner, and common failure mode.

Layer	Lifetime	Access	Primary owner	Failure mode
Live context C_{live}	Current turn	Already visible in the prompt window	Context assembler	Critical evidence left out of view
Retrievable memory M	Current workflow	$R(q, M)$ fetch by query or ID	Memory store	Low recall or wrong retrieval
Long-term memory M_{long}	Across sessions	Profile/project/history recall	Product memory service	Stale facts, privacy boundaries
Cache K	Until invalidated	Keyed reuse of computed results	Runtime/cache layer	Stale or polluted results

Rule of thumb: visibility serves immediate reasoning; memory serves recoverability; cache serves reuse.

图 13: 实时上下文、可回溯上下文、长期记忆和缓存的对照表。关键差异在生命周期、访问路径、责任模块和典型失效模式。

可以把四类机制想成四种“信息位置”。实时上下文像书桌上摊开的纸，模型马上能看见；可回溯上下文像书桌旁边贴好标签的文件夹，需要查询后取回；长期记忆像跨项目的档案柜，新会话也能找到；缓存像复用已经算过的中间结果，目标是省时间或省成本。

- 实时上下文：也就是当前进入上下文窗口的 C_{live} ，生命周期最短，影响最直接，失败模式是关键材料没有被放到模型眼前。
- 可回溯上下文：来自记忆存储 M ，通过 $R(q, M)$ 按问题取回，适合保存被移出窗口的中段历史、工具结果和冗长载荷，失败模式是召回不到或召回错材料。
- 长期记忆：记为 M_{long} ，跨新会话保留用户、项目和先前讨论的可延续线索，失败模式是过期事实、隐私边界和错误泛化。
- 缓存：记为 K ，偏向复用已有计算、检索结果或中间数据，失败模式是缓存污染、命中过期结果，或把“算得快”误当成“记得对”。

演讲者在问答里说得很坦率：他们目前更关注长期记忆，还没有深入缓存处理。原因并不玄妙，投诉集中在连续性上。用户抱怨的核心常常是“Alex 明明之前聊过，为什么新开会话就像没发生过”。这类问题靠缓存很难解决，因为缓存解决的是复用，长期记忆解决的是身份、项目、目标和历史承诺的延续。

6.3 上下文预算：先分配窗口，再填充内容

团队承认，上下文选择目前仍然很启发式。比如保留前 100 条和后 100 条，这种启发式策略简单、可实现、容易解释，却无法保证“留下来的就是正确内容”。像整理行李一样，只按“最早几件”和“最新几件”打包，可能会漏掉中间那份护照；代理系统里漏掉的可能是安全约束、用户偏好、关键工具输出或某个失败跨度。

因此，下一步需要把 token 预算从一个总量，拆成可讨论、可测试、可调参的上下文预算：

$$B = B_{\text{sys}} + B_{\text{hist}} + B_{\text{tools}} + B_{\text{ret}} + B_{\text{ans}}$$

- B ：本轮可用的总 token 预算。
- B_{sys} ：分配给系统提示和不可丢失约束的预算。
- B_{hist} ：分配给对话历史的预算，包括头尾保留后的用户目标、决策和近期上下文。
- B_{tools} ：分配给工具调用结果的预算，尤其是最新、最可信、最能支撑下一步行动的结果。
- B_{ret} ：分配给可回溯上下文的预算，即从记忆存储或长期记忆中取回的材料。
- B_{ans} ：预留给模型生成回答、计划或工具参数的输出预算。

上下文预算的工程价值在于把“塞不塞得下”升级为“为什么这一类信息该拿这么多空间”。如果 B_{tools} 太小，代理可能忽略刚查到的证据；如果 B_{ret} 太大，历史回忆会挤掉当前任务；如果 B_{ans} 没有预留，模型会在回答末尾突然收缩，造成计划残缺或工具参数不完整。

6.4 上下文质量评估：别只改提示词，要测上下文

演讲者最后给出的经验很硬：很多被归因于提示词的问题，根上是上下文问题。早期大家会盯着提示工程，以为把措辞调得更精密就能修复失败；Alex 的实践显示，模型经常失败在“看错了材料”或“没看到材料”。提示词仍然重要，但提示词无法替代证据选择。

这也是 EBO 和第 $N+1$ 轮测试的意义：它们把“长会话里有没有忘掉关键事实”变成可观察的问题。团队还没有成熟的上下文质量评估标准，但已经在问更好的问题：保留的上下文是否正确？被检索出来的记忆是否相关？子代理返回的紧凑结果是否足以让主代理继续工作？这些问题合在一起，才构成 $Q(C, q)$ 的工程目标。

常见误区是把上下文窗口当成垃圾桶：只要模型支持更长输入，就不断追加历史、追踪数据和工具结果。这样做会让重要信息沉在噪声里。对代理来说，信息过多和信息不足都会失败，差别只在于失败时看起来更“努力”。

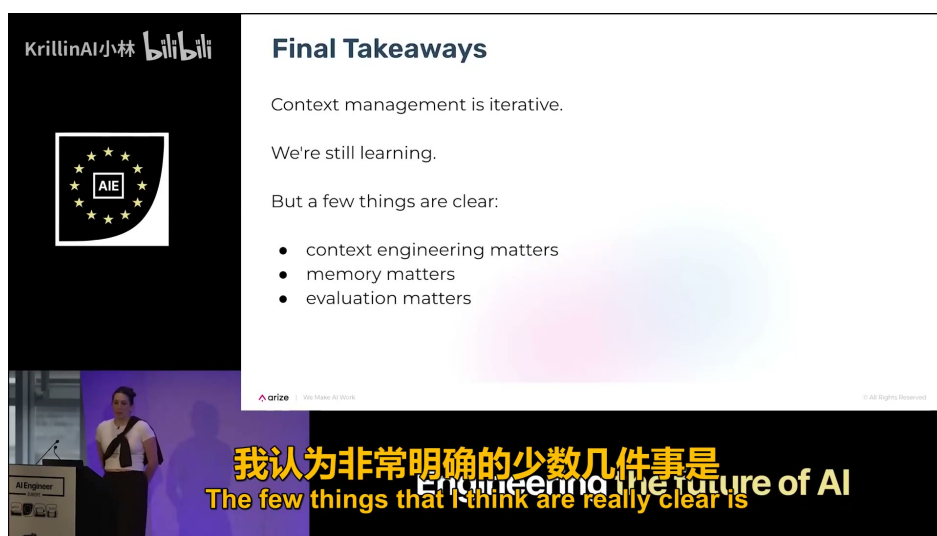


图 14：最终总结把上下文工程、记忆机制和评估指标列为三条主线，提示构建者要把上下文质量纳入工程验证。源视频区间：00:13:50–00:14:18。

6.5 Claude Code 线索与问答中的实现细节

演讲者提到，Claude Code 相关材料让他们看到类似的截断和压缩方向，这给了团队某种验证：他们遇到的问题不是 Alex 独有，其他代码代理也在处理相近的上下文压力。不过那些材料没有给出完整秘诀，所以团队仍然要继续摸索自己的长期记忆、预算和评估方法。

问答部分补上了一个很具体的实现侧面：当前长期记忆工作大致把记忆项存进数据库，为它们分配 ID，让 Alex 拥有一个可以看到这些 ID 和对应对话位置的工具；对于早期消息，系统会提供部分内容预览。换成数据结构的想象，就是建立一个 $id_i \mapsto m_i$ 的 ID 化记忆索引，再让代理根据任务需要定位和取回材料。



图 15: 问答环节补充了实现取舍：团队当前更关注长期记忆，缓存处理还没有深入展开，原因是用户反馈集中在跨会话连续性。源视频区间：00:15:18–00:15:46。

ID 化记忆索引的好处是边界清楚：实时上下文不必携带所有历史正文，只携带可引用的索引、位置和预览。它像目录卡片，先告诉代理“档案柜里有哪些相关抽屉”，再允许它按需打开。这样做仍有风险，例如 ID 描述过粗、预览误导检索、旧记忆需要更新，但它比把整段历史塞进窗口更可控。

演讲者也承认，这套实现还需要更复杂的设计和资源投入；只是目前运行正常，而最痛的用户反馈集中在长期记忆，所以团队先把精力放在那里。这个取舍很现实：工程优先级不由概念新鲜度决定，而由用户失败模式决定。

6.6 本章小结

这一章收束出三条主线。第一，上下文工程是代理可靠性的底层问题，因为模型行为受它实际看见的材料约束。第二，记忆机制要分层：实时上下文管即时推理，可回溯上下文管本轮历史恢复，长期记忆管跨会话连续性，缓存管可复用结果。第三，评估必须进入长会话和上下文选择本身，否则团队只会在提示词表面打磨，错过真正的失败来源。

给代理构建者的可执行检查清单如下：

1. 先定义上下文预算，再填充上下文窗口。
2. 保护系统提示和最新工具调用结果，避免它们被历史噪声挤掉。
3. 把中段历史、冗长载荷和旧工具结果存成带 ID 的可回溯材料。
4. 用第 N+1 轮测试检查长会话里的上下文状态。
5. 把重数据任务委托给子代理，让主代理只接收紧凑结果。
6. 衡量上下文质量评估，别只依赖提示词修改来解释或修复失败。